

Prova di programmazione (12 punti)

1. 2 punti

Sono dati due vettori di interi a , b (non ordinati e contenenti eventualmente dati ripetuti), di dimensione rispettivamente na e nb . Si scriva una funzione che verifichi se il primo vettore sia una sotto-sequenza del secondo. La funzione deve poter essere chiamata come

```
ris = subSeq(a, na, b, nb);
```

2. 4 punti

È dato un BST avente come valori e chiavi delle stringhe. Si scriva una funzione che determini il cammino radice-foglia contenente il massimo numero di nodi aventi 2 figli (non nulli/vuoti). La funzione stampi il cammino e ne ritorni la lunghezza. Nel caso di più cammini di lunghezza massima la scelta è arbitraria. La funzione abbia prototipo:

```
int BSTprintMax2(BST b);
```

Si richiede, oltre alla funzione, la definizione del tipo *BST* e del tipo usato per il nodo.

3. 6 punti

Due stringhe $s1$ e $s2$ possono essere concatenate se il prefisso (proprio) di ordine I di $s2$ coincide col suffisso (proprio) di ordine I di $s1$. Si ricorda che il prefisso (suffisso) di ordine I di una stringa è una sottostringa iniziale (terminale) di I elementi, che si dice proprio se non coincide con la stringa stessa (I è minore della lunghezza della stringa). Si scriva una funzione **checkConcat** che, date due stringhe, verifichi se possono essere concatenate. Si scriva poi una funzione che, dato un insieme S di stringhe, trovi la più lunga sequenza di stringhe concatenabili, appartenenti all'insieme. L'insieme S è

rappresentato come vettore. Non sono ammesse ripetizioni. Come lunghezza (da massimizzare) si intende il conteggio delle stringhe nella sequenza. La funzione abbia prototipo:

```
int maxConcatSeq(char **S, int n);
```